

SCHOOL OF INFORMATION TECHNOLOGY

SHRI RAMASAMY MEMORIAL UNIVERSITY SIKKIM

TADONG -737102



NAME : 1. Madhav Bhandari (24IT102040)
REGISTER NO : 2. Phurba Lhamu Tamang (24IT102026)
3. Pema Wangyel (24IT102049)
4. Tshering Dorji Tamang (24IT102037)
5. Ugyen Chophel (24IT102050)
PROGRAM : BCA
NAME & COURSE CODE : Distributed System (BCA23CO51)
ACADEMIC YEAR : 2026
SUBMITTED TO : Mrs. Sumitra Das

Signature of subject incharge

Table of Contents

Table of Contents	2
1. Introduction.....	3
2. Problem Statement	3
3. Project Objectives	3
4. Technical Feasibility	3
Required Technologies	4
Technical Working.....	4
Technical Risks and Solutions	4
5. Economic Feasibility	5
Cost Estimation	5
6. Operational Feasibility	5
User Operations	6
7. Schedule Feasibility	6
8. Legal and Ethical Feasibility.....	6
9. Storage Feasibility	7
Deduplication.....	7
Replication Factor	7
Erasure Coding.....	7
10. Risk Assessment	7
11. Conclusion	8

1. Introduction

A Distributed File Storage System is a software system that stores files across multiple storage nodes instead of keeping every file in one central server. When a user uploads a file, the system divides it into small pieces called chunks. These chunks are stored in different storage locations. The system keeps metadata records that identify each chunk, its location, its sequence, and its hash value.

When a user downloads a file, the system retrieves the required chunks from storage nodes, verifies that they are correct, arranges them in the original sequence, and reconstructs the complete file.

The project applies important distributed-systems concepts, including file chunking, metadata management, hashing, replication, fault tolerance, and node monitoring.

2. Problem Statement

Traditional file-storage systems often store all files on one computer or server. This approach has several limitations:

- If the main server fails, files may become inaccessible.
- The central server can run out of storage space.
- A large number of users can reduce system performance.
- A hardware failure can result in data loss.
- The system has a single point of failure.

The proposed Distributed File Storage System addresses these problems by splitting a file into chunks and storing the chunks across different nodes. The system also maintains chunk-location information and backup copies to improve reliability.

3. Project Objectives

The main objectives of the project are:

- To allow users to register and log in securely.
- To allow users to upload files to the system.
- To divide large files into smaller chunks.
- To calculate a hash value for each chunk.
- To distribute file chunks among available storage nodes.
- To maintain metadata about files, chunks, users, and storage-node locations.
- To create chunk replicas for improved availability.
- To reconstruct the original file during download.
- To verify file integrity before providing the file to the user.
- To allow an administrator to monitor nodes and system operations.

4. Technical Feasibility

Technical feasibility checks whether the required technology, hardware, software, skills, and development tools are available to build the system.

The DFSS project is technically feasible because it can be developed using standard programming tools, open-source frameworks, a relational database, and existing student computers. The project can simulate distributed storage using different folders, Docker containers, virtual machines, or separate computers connected through a local network.

Required Technologies

Component	Suggested Technology
Front end	HTML, CSS, JavaScript, Bootstrap, React
Back end	Python Flask, FastAPI, Django, Node.js, Express.js
Database	MySQL, PostgreSQL, SQLite
Hashing	SHA-256
Storage nodes	Local folders, Docker containers, virtual machines, separate computers
API testing	Postman
Version control	Git and GitHub
Diagram tool	draw.io

Technical Working

- The user uploads a file.
- The system validates the uploaded file.
- The file is divided into chunks.
- A SHA-256 hash value is generated for each chunk.
- The system selects available storage nodes.
- Chunks and replica copies are stored on nodes.
- Metadata is saved in the database.
- During download, chunks are retrieved, verified, arranged in order, and combined.

The project is realistic because the first version can use a small number of storage nodes, such as three local folders or three Docker containers. More advanced features can be added after the basic version works.

Technical Risks and Solutions

Technical Risk	Proposed Solution
A storage node becomes unavailable	Store replicas on other nodes
A file chunk is corrupted	Use SHA-256 hash verification
Incorrect chunk order	Store a chunk index for each file

Technical Risk	Proposed Solution
Database metadata is missing	Keep database backup and validate records
One node becomes full	Select another node using available-capacity information
Network interruption	Display proper error messages and retry failed operations

Therefore, the project is technically feasible.

5. Economic Feasibility

Economic feasibility determines whether the expected benefits of the project are reasonable compared to its cost.

The DFSS project is economically feasible because it mainly uses free and open-source software. The prototype does not require expensive servers or paid cloud services. Existing computers can act as storage nodes during testing and demonstration.

Cost Estimation

Item	Estimated Cost
Programming language and frameworks	Free
Database software	Free
VS Code or other code editor	Free
GitHub repository	Free
Postman API testing tool	Free
draw.io diagram tool	Free
Local storage-node simulation	Free
Cloud hosting	Optional
Development time	Main project cost

The major resource required is the time and effort of the project team. Therefore, the project has low cost and is suitable for a semester-end academic project.

6. Operational Feasibility

Operational feasibility checks whether the system can be used effectively by its intended users.

The DFSS is operationally feasible because it follows simple file-management operations that users already understand. A user only needs to log in, upload a file, see uploaded files, download a file, or delete a file.

User Operations

User Type	Main Operations
Normal User	Register, log in, upload file, view files, download file, delete file, log out
Administrator	View users, check storage-node status, monitor storage capacity, review system logs

The application will provide a clean dashboard, clear error messages, success notifications, and simple buttons. Users do not need to understand the internal chunking or distributed-storage process to use the system.

Therefore, the project is operationally feasible.

7. Schedule Feasibility

Schedule feasibility checks whether the system can be completed within the available academic time.

The DFSS project can be developed gradually. The team should first complete the essential functions, then add advanced features if time remains.

Phase	Activities
Week 1	Requirement analysis and project planning
Week 2	Feasibility study, flowchart, ER diagram, database design
Week 3	User registration and login module
Week 4	File upload and metadata-storage module
Week 5	File chunking and SHA-256 hashing
Week 6	Storage-node allocation and chunk storage
Week 7	File reconstruction and download
Week 8	Replication and node-status monitoring
Week 9	Testing, debugging, and security checks
Week 10	Documentation, presentation, and final demonstration

The minimum working version should include user login, file upload, chunking, metadata storage, and file download. Replication, deduplication, and automatic recovery may be developed as extra features.

Therefore, the project is schedule feasible.

8. Legal and Ethical Feasibility

Legal and ethical feasibility ensures that the system is developed and used responsibly. The project should follow these rules:

- Users must upload only files they are authorized to use.

- The system should not be used for illegal file sharing.
- Passwords must not be stored in plain text.
- Passwords should be stored using secure hashing.
- Users should only access their own files.
- Administrators should have separate roles and permissions.
- Sensitive personal or confidential files should not be used in the classroom demonstration.
- The project should use properly licensed open-source libraries.

The system should maintain user privacy and protect stored data from unauthorized access.

Therefore, the project is legally and ethically feasible for academic use.

9. Storage Feasibility

A key concern in distributed storage is that replication can increase storage usage. If every file chunk is stored in two or three locations, total physical storage required will increase.

The DFSS can reduce unnecessary storage use through the following methods:

Deduplication

Deduplication checks whether the same chunk already exists in the storage system. If the chunk is already stored, the system can reuse the existing chunk instead of creating another identical copy. This is especially useful when multiple users upload similar files or different versions of the same document.

Replication Factor

The project can use a replication factor of two. This means every chunk is stored on two different nodes. For example:

- Original file size: 100 MB
- Replication factor: 2
- Total physical storage required: approximately 200 MB

Replication improves availability but increases storage use.

Erasure Coding

Erasure coding is an advanced future improvement. It divides data into data chunks and parity chunks. It provides fault tolerance with less storage overhead than storing multiple complete replicas.

For the classroom prototype, the team can use replication. For the report and future work section, the team can mention deduplication and erasure coding as methods to improve storage efficiency. A previous project discussion identified (4,2) erasure coding as a possible approach with roughly 1.5 times storage overhead while tolerating the loss of two encoded chunks.

10. Risk Assessment

Risk	Impact	Solution
Storage node failure	Users cannot retrieve some chunks	Store chunk replicas on different nodes
Data corruption	Downloaded file may be damaged	Verify SHA-256 hash values
Unauthorized access	User files may be exposed	Use authentication and role-based access
Storage-node capacity becomes full	Upload may fail	Monitor available storage and select another node
Metadata loss	Chunk locations may be unknown	Back up the metadata database
Limited development time	Advanced features may remain incomplete	Focus first on core functions
Network failure	Node communication may fail	Use retry handling and error messages

11. Conclusion

The Distributed File Storage System is feasible for development as an academic software project. It is technically possible to build using free tools, economical because it requires low cost, operationally simple for users, achievable within the academic schedule, and legally suitable when proper security and ethical guidelines are followed.

The system provides a practical solution to the limitations of centralized file storage. It demonstrates core distributed-systems ideas such as file chunking, metadata management, hashing, replication, fault tolerance, and storage-node monitoring. The recommended decision is to proceed with the project development.